Característica 0006: cambios para completar la entrega y llegar al nivel 4

Redirect cache with cross-replica invalidation · módulo propietario `links` · peso 21 · 17 de octubre de 2026

> El peso 21 exige nivel 4: un evento nuevo del equipo, que cruce de módulo, externalizado a un broker durable. Reutilizar `ShortUrlCreatedEvent` o `ClickLoggedEvent` no llega a ese nivel. Redis Pub/Sub tampoco: si una réplica no está escuchando, el mensaje se pierde.

1. Qué había que cumplir

La entrega de la característica 0006 y la prueba de concepto del 23 de octubre pedían, juntas, esto:

- Los módulos ya existentes siguen conectados.
- Una característica nueva publica un evento y otro módulo lo consume. Ese cruce tiene que verse en un test.
- Hay un ADR de la característica a la que pertenece el evento.
- `docker compose up` arranca sin errores.
- La entrega a Moodle es un zip del proyecto.
- Al ser peso 21, el evento tiene que ser de nivel 4: broker externo (`--profile broker`), paralelo, aislado por módulo y durable.

El README del proyecto lo fija así: el nivel 3 es un evento Modulith en proceso (`@ApplicationModuleListener`) y el nivel 4 es un broker externo. `--profile broker` solo arrancaba RabbitMQ; las aplicaciones no estaban conectadas.

2. Punto de partida

La primera iteración de la caché ya hacía una parte de la característica:

- `GET /{hash}` guarda destino en caché. Los hash desconocidos o inactivos no se guardan.
- En Docker la caché es Redis, con TTL `urlshortener.links.redirect-cache-ttl`. En local es una caché en memoria.
- `POST /api/link/{hash}/deactivate` exige `X-Admin-Token`, persiste la baja y es idempotente.
- `links` publicaba `ShortUrlUpdatedEvent`, pero lo consumían solo listeners del propio módulo para borrar la caché.
- El ADR 0004 ya existía y rechazaba el broker externo.

Eso no bastaba para la entrega ni para el nivel 4. El evento no cruzaba de módulo, `./gradlew check` estaba en rojo por el estilo de `build.gradle.kts`, y la invalidación entre réplicas dependía de Redis Pub/Sub, que es transitorio.

3. Por qué cada cambio

3.1. Que el evento cruce de módulo

`analytics` es quien posee el modelo de lectura por hash. Si la baja se quedaba dentro de `links`, `GET /api/stats/{hash}` seguía describiendo un enlace desactivado como activo. La caché no se movió de módulo: sigue siendo de `links`. El acoplamiento permitido es el evento.

- `LinkStats` gana el campo `active`. El total de clics no se toca.
- `LinkStatsListener` sigue escuchando el alta y el clic en proceso. La baja, sin broker, la atiende `InProcessShortUrlUpdatedListener`.
- `GET /api/stats/{hash}` devuelve `active`.
- `LinkDeactivationTests` espera a que, tras desactivar, las estadísticas sigan con los mismos clics y pasen a `active: false`. También comprueba que un hash desconocido o inactivo no queda en caché.

Eso cubre la prueba de concepto: evento nuevo, publicado por `links`, consumido por `analytics`, con test y con el ADR 0004.

3.2. Que la invalidación de la caché compartida no dependa de una carrera

`RedisShortUrlUpdatedListener` borra la clave `urlshortener:redirect:{hash}` en el mismo callback posterior al commit y, además, publica en Redis Pub/Sub. La clave es compartida, así que el borrado lo ven las dos réplicas antes de que vuelva la respuesta HTTP. Pub/Sub repite el borrado por si otra réplica vuelve a escribir la clave mientras el evento viaja. Este canal se mantiene para `docker compose up` sin broker. No es el transporte de nivel 4.

3.3. Nivel 4: broker externo, durable y aislado por módulo

Con el perfil `broker`, `ShortUrlUpdatedEvent` lleva `@Externalized("urlshortener.short-url-updated::links.ShortUrlUpdatedEvent")`. Spring Modulith lo envía a RabbitMQ después del commit. Si la publicación falla, el registro JDBC de Modulith la reintenta. Los mensajes se marcan persistentes y el exchange es durable.

Cada módulo tiene su propia cola durable, enlazada a ese exchange. Las dos réplicas consumen cada cola. Eso es el aislamiento de módulos y el paralelismo que pide el nivel 4:

Cola durable	Módulo	Qué hace al recibir el evento
`urlshortener.links.redirect-invalidation`	`links`	Borra la entrada de la caché de redirect.
`urlshortener.analytics.short-url-updated`	`analytics`	Pone `LinkStats.active` en false sin cambiar el total de clics.

Si una réplica está caída, el mensaje espera en la cola. Redis Pub/Sub no puede hacer eso. Por eso el ADR deja de rechazar el broker y lo adopta como transporte del evento. Redis sigue siendo el almacén de la caché, no el bus del evento.

Sin el perfil `broker`, la externalización está apagada y `analytics` consume la baja en proceso. Así los tests locales y `docker compose up` no necesitan RabbitMQ. Con el perfil `broker`, la baja de estadísticas solo entra por la cola: el test puede demostrar que el broker es quien entrega el evento.

El arranque lo activa solo. El `Dockerfile` usa el perfil `docker` y, si el nombre `rabbit` resuelve en la red de Compose, añade `broker`. `docker compose --profile broker up` crea RabbitMQ. Las aplicaciones esperan a que esté sano cuando ese servicio existe; si no se pide el perfil, la dependencia es opcional y el producto de siempre sigue arrancando.

3.4. Puerta de ingeniería y papeles de la entrega

- Se corrigió la indentación de la dependencia Redis en `build.gradle.kts`. Era lo que hacía fallar ktlint y, con ello, `./gradlew check`.
- El paquete de caché queda marcado como adaptador secundario, igual que persistencia y validación.
- El ADR 0004, la ficha 0006 y `AGENTS.md` describen el consumidor de `analytics` y la externalización a RabbitMQ.
- La ficha declara el uso de Copilot y del agente de Cursor. Sin esa divulgación, la nota de ingeniería de la característica es 0.

4. Ficheros tocados

Fichero	Para qué
`links/ShortUrlUpdatedEvent.kt`	Anota el evento para externalizarlo al exchange y la routing key.
`links/adapters/broker/RedirectInvalidationBroker.kt`	Exchange, cola durable de `links`, mensajes persistentes y consumidor que borra la caché.
`analytics/adapters/broker/ShortUrlUpdatedBrokerListener.kt`	Cola durable de `analytics` y consumidor que marca el enlace inactivo.
`analytics/adapters/events/LinkStatsListener.kt`	Alta y clic siguen en proceso. La baja en proceso solo existe si no hay perfil `broker`.
`analytics` application, persistence y web	Campo `active`, upsert que no reactiva un enlace ya dado de baja, y respuesta de `/api/stats`.
`links/adapters/cache/RedisShortUrlUpdatedListener.kt`	Borrado inmediato de la clave compartida, además de Pub/Sub.
`links/adapters/cache/package-info.java`	El paquete de caché queda como adaptador secundario.
`application.yaml`	Externalización apagada por defecto y encendida, en serie, con el perfil `broker`.
`build.gradle.kts` y `libs.versions.toml`	AMQP, `spring-modulith-events-amqp` y Testcontainers de RabbitMQ. Indentación que rompía ktlint.
`Dockerfile` y `docker-compose.yml`	El perfil `broker` se activa cuando RabbitMQ está en la red. La dependencia es opcional.
`LinkDeactivationTests.kt`	Token, idempotencia, caché y consumo de `analytics` sin broker.
`ShortUrlUpdatedBrokerTests.kt` y `RabbitContainer.kt`	Con RabbitMQ real, las estadísticas solo pasan a inactivas cuando la cola entrega el evento.
`docs/adr/0004-redis-redirect-cache.md`	Decisión de externalizar el evento y de no usar Pub/Sub como transporte de nivel 4.

`docs/features/0006-redirect_cache_cross_replica.md`	Criterios, evidencia de las dos réplicas y del broker, y divulgación de IA.
`AGENTS.md`	El mapa de eventos incluye `ShortUrlUpdatedEvent` y el camino de nivel 4.

5. Qué se ejecutó

Comprobación	Resultado
`./gradlew check`	**Verde. Incluye ktlint, detekt, tests y JaCoCo.**
`LinkDeactivationTests`	**401 sin token, baja idempotente, 404, caché vacía para hash desconocido o inactivo, y `active: false` en estadísticas.**
`ShortUrlUpdatedBrokerTests`	**Con perfil `broker` y RabbitMQ, `analytics` recibe la baja por la cola.**
`docker compose up`	**`db`, `redis`, `app1`, `app2` y `lb` sanos. Tras desactivar, las dos réplicas responden 404 y la clave Redis desaparece.**
`docker compose --profile broker up`	**Perfiles activos `docker` y `broker`. Dos colas durables, cada una con un consumidor por réplica. Mismo resultado de 404 y `active: false`.**

En la pasada con broker, las colas quedaron así: `urlshortener.links.redirect-invalidation` y `urlshortener.analytics.short-url-updated`, las dos durables, con 2 consumidores y 0 mensajes pendientes después de procesar la baja.

6. Cómo queda cubierta la rúbrica

Requisito	Dónde se cumple
Evento nuevo del equipo	`ShortUrlUpdatedEvent`. No se reutiliza un evento de las características de partida.
Cruza de módulo	`links` publica. `analytics` consume. `LinkDeactivationTests`.
ADR	`docs/adr/0004-redis-redirect-cache.md`.
Broker externo	RabbitMQ del perfil `broker`, vía Spring Modulith AMQP.
Durable	Exchange y colas durables, mensajes persistentes, reintento del registro de publicación.
Aislado por módulo	Una cola por módulo. `links` no llama a `analytics`.
Paralelo	Las dos réplicas consumen las dos colas a la vez.
`docker compose up`	Sigue arrancando sin RabbitMQ.
`./gradlew check` y divulgación de IA	Verdes en la ficha. Sin las dos, la ingeniería de la característica vale 0.
Zip de Moodle	`C:\Users\juano\Project-Web\urlshortener-0006.zip`. Hay que subirlo en Moodle; desde el repositorio no se envía.

7. Comandos de la demo

Producto sin broker, el que pide que Compose arranque:

```
docker compose up --build
```

Nivel 4, con el evento en RabbitMQ:

```
$env:URLSHORTENER_ADMIN_TOKEN = "demo-token"
docker compose --profile broker up --build
```

El token no está en el repositorio. Si queda vacío, desactivar responde 401. Para parar la pila: `docker compose --profile broker down` dentro de `urlshortener-main\urlshortener-main`.